ЛАБОРАТОРНА РОБОТА № 2

Unit тестування асинхронного коду з Jest

Мета: Навчитися тестувати асинхронні функції TypeScript за допомогою Jest і ts-jest. Узагальнити функції першої лабораторної через generics, застосувати типізовані mock-функції, матчери, lifecycle hooks та fake timers. Перевірити успішні сценарії, помилки і граничні значення та виміряти покриття вихідного коду.

Хід роботи

1 Перевірка середовища

Для роботи використано встановлені Node.js і npm. Версії середовища та прямі залежності перевірено у PowerShell. Список npm ls --depth=0 показує пакети самого проєкту без повного дерева їхніх залежностей.

```
node --version
npm.cmd --version
npm.cmd ls --depth=0
```

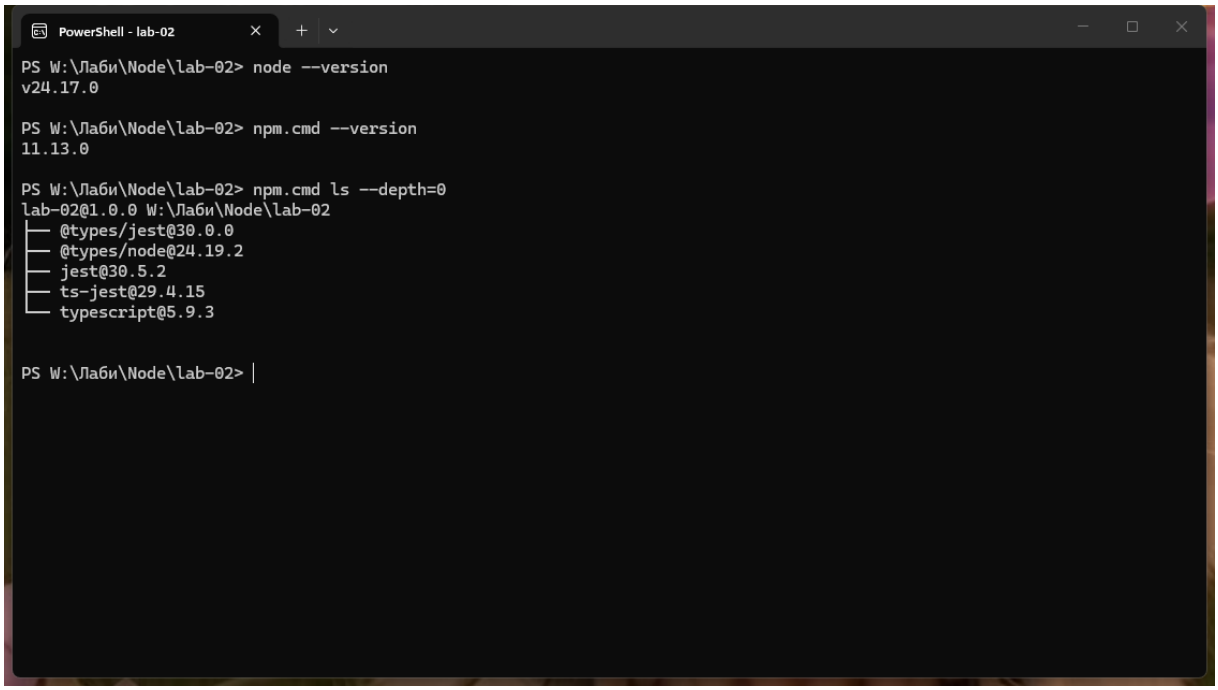


Рис. 1. Версії Node.js, npm та залежностей проєкту

Компонент	Версія
Node.js	24.17.0
npm	11.13.0
TypeScript	5.9.3
Jest	30.5.2
ts-jest	29.4.15
@types/jest	30.0.0
@types/node	24.19.2

ts-jest перетворює TypeScript перед виконанням тестів. Для встановленої версії ts-jest заявлено діапазон TypeScript >=4.3 <7, тому в цьому проєкті зафіксовано TypeScript 5.9.3. Версія компілятора першої лабораторної не змінювалася.

2 Залежності та сценарії запуску

Створено окремий проєкт lab-02. Додано jest, ts-jest, @types/jest, TypeScript та типи Node.js як devDependencies. Файл package-lock.json фіксує точні версії всього дерева залежностей. Для звичайного встановлення тестових пакетів використовується наведена команда [1].

```
npm.cmd install -D jest ts-jest @types/jest
```

Після першого встановлення перевірено відтворення залежностей командою npm ci --offline. Вона встановила 292 пакети з локального кешу відповідно до lock-файла. На іншому комп’ютері, де кеш ще порожній, потрібно виконати npm ci з доступом до мережі.



Рис. 2. Повторне встановлення залежностей за lock файлом

Попередження deprecated для транзитивного glob не зупинило встановлення. Воно стосується залежності інструментів тестування. Успішність налаштування додатково перевірено компіляцією та запуском Jest.

Лістинг package.json

```
{
  "name": "lab-02",
  "version": "1.0.0",
  "private": true,
  "description": "Лабораторна робота 2. Unit-тестування асинхронного коду з Jest",
  "author": "Сокирко Владислав Володимирович, ІПЗ-23-4",
  "type": "commonjs",
  "main": "dist/async-utils.js",
  "scripts": {
    "build": "tsc -p tsconfig.build.json",
    "typecheck": "tsc --noEmit",
    "test": "jest --runInBand",
    "test:watch": "jest --watch",
    "test:coverage": "jest --coverage --runInBand"
  },
  "devDependencies": {
    "@types/jest": "^30.0.0",
    "@types/node": "^24.19.2",
    "jest": "^30.0.0",
    "ts-jest": "29.4.15",
    "typescript": "5.9.3"
  }
}
```

Сценарій	Призначення
typecheck	Перевіряє типи вихідного коду і всіх тестів без генерації JavaScript.
build	Компілює вихідний модуль і створює декларації типів у dist.
test	Запускає всі набори Jest в одному процесі через --runInBand.

Сценарій	Призначення
test:watch	Повторює тести після змін у Git-репозиторії.
test:coverage	Запускає тести та перевіряє встановлені пороги покриття.

Для watch-режиму до створення Git-репозиторію можна виконати npm run test:watch -- --watchAll. У звичайному та coverage-запусках використано --runInBand, щоб не створювати окремі worker-процеси для невеликого навчального проєкту.

3 Налаштування TypeScript та Jest

Проєкт використовує CommonJS, тому jest.config.js має форму module.exports із методички. Компілятор перевіряє як src, так і test у строгому режимі. noUncheckedIndexedAccess враховує можливий undefined під час доступу до елементів за індексом.

Лістинг tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "CommonJS",
    "moduleResolution": "Node",
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "esModuleInterop": true,
    "noEmitOnError": true,
    "forceConsistentCasingInFileNames": true,
    "skipLibCheck": true,
    "types": ["node", "jest"]
  },
  "include": ["src/**/*.ts", "test/**/*.ts"],
  "exclude": ["node_modules", "dist", "coverage"]
}
```

Окрема build-конфігурація компілює лише робочий модуль. Тестові файли залишаються TypeScript і виконуються через ts-jest; до dist вони не потрапляють.

Лістинг tsconfig.build.json

```
{
  "extends": "./tsconfig.json",
  "compilerOptions": {
    "rootDir": "src",
    "outDir": "dist",
    "declaration": true,
    "types": ["node"]
  },
  "include": ["src/**/*.ts"],
  "exclude": ["node_modules", "dist", "coverage", "test"]
}
```

Лістинг jest.config.js

```
module.exports = {
  preset: 'ts-jest',
  testEnvironment: 'node',
  reporters: ['default'],
  testMatch: ['**/test/**/*.test.ts'],
  collectCoverageFrom: ['src/**/*.ts'],
  coverageDirectory: 'coverage',
  coverageReporters: ['text', 'html', 'lcov', 'json-summary'],
  coverageThreshold: {
    global: {
      statements: 100,
      branches: 100,
      functions: 100,
      lines: 100,
    },
  },
};
```

preset: ts-jest забезпечує обробку TypeScript, а testEnvironment: node задає середовище Node.js. testMatch знаходить файли *.test.ts. collectCoverageFrom включає весь src/**/*.ts, зокрема функцію delay. Пороги coverageThreshold дорівнюють 100: команда завершиться з помилкою, якщо хоча б один показник буде нижчим [2].

reporters: [default] явно задає стандартний вивід Jest. Параметр --verbose під час окремих запусків показує назву й результат кожної перевірки.

Звіти text, html, lcov і json-summary дають таблицю в терміналі та файли в coverage. Папки node_modules, dist і coverage внесено до .gitignore, оскільки вони відтворюються командами проєкту.

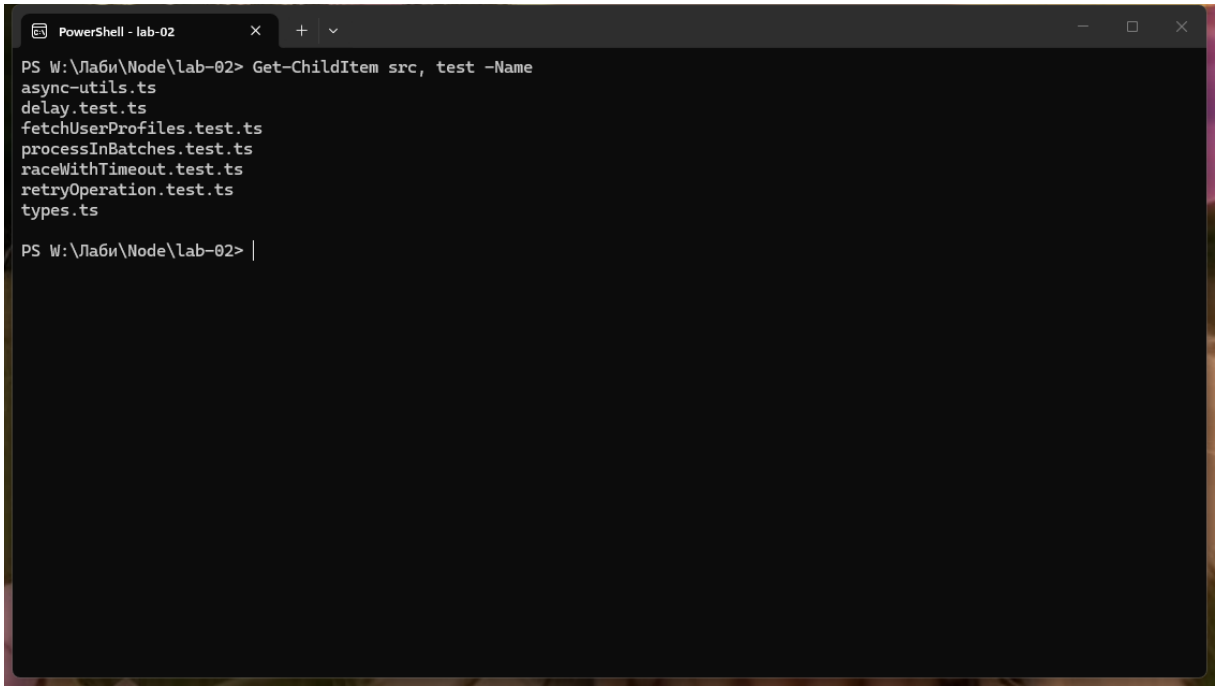


Рис. 3. Вихідний модуль та п’ять файлів тестів TypeScript

4 Узагальнення функцій через generics

Порівняно з першою лабораторною змінено типові параметри й анотації трьох функцій. Алгоритми, затримка 100 мс між спробами, послідовність партій та текст помилки таймауту збережені. Порівняння JavaScript після видалення TypeScript-анотацій підтвердило однаковий виконуваний код обох модулів.

Функція	**Зв'язок типів**
retryOperation<T>	operation повертає Promise<T>; функція повертає Promise<T>.
processInBatches<T, R>	На вході T[]; processor перетворює T[] на R[]; результат R[].
raceWithTimeout<T>	Тип значення початкового Promise<T> зберігається в результаті.

Лістинг retryOperation

```
export async function retryOperation<T>(
  operation: () => Promise<T>,
  maxRetries: number = 3,
): Promise<T> {
  if (!Number.isInteger(maxRetries) || maxRetries < 1) {
    throw new RangeError('maxRetries must be a positive integer');
  }

  for (let attempt: number = 1; ; attempt++) {
    console.log(`Спроба ${attempt}...`);
    try {
      return await operation();
    } catch (error: unknown) {
      if (attempt === maxRetries) {
        throw error;
      }
      await delay(100);
    }
  }

}
```

maxRetries означає загальну кількість спроб, за замовчуванням 3. Після успіху результат повертається відразу. Після останньої невдачі передається та сама помилка, без додаткової паузи. Цикл завершується через return або throw; зайвого недосяжного коду після нього немає.

Лістинг processInBatches

```
export async function processInBatches<T, R>(
  items: T[],
  batchSize: number,
  processor: (batch: T[]) => Promise<R[]>,
): Promise<R[]> {
  if (!Number.isInteger(batchSize) || batchSize < 1) {
    throw new RangeError('batchSize must be a positive integer');
  }

  const results: R[] = [];
  const total: number = Math.ceil(items.length / batchSize);

  for (let offset: number = 0; offset < items.length; offset += batchSize) {
    const batchNumber: number = Math.floor(offset / batchSize) + 1;
    const batch: T[] = items.slice(offset, offset + batchSize);
    console.log(`Обробка партії ${batchNumber}/${total}...`);

    const processed: R[] = await processor(batch);
    for (const value of processed) {
      results.push(value);
    }
  }

  return results;
}
```

Типи T і R можуть відрізнятися: наприклад, масив чисел перетворюється на масив рядків. await processor(batch) завершує поточну партію перед запуском наступної. Перша й остання неповна партії проходять той самий алгоритм обробки.

Лістинг raceWithTimeout

```
export async function raceWithTimeout<T>(
  promise: Promise<T>,
  timeoutMs: number,
): Promise<T> {
  if (!Number.isFinite(timeoutMs) || timeoutMs < 0 || timeoutMs > 2_147_483_647) {
    throw new RangeError('timeoutMs must be between 0 and 2147483647');
  }

  let timer: ReturnType<typeof setTimeout> | undefined;
  const timeout: Promise<never> = new Promise<never>((_resolve, reject): void => {
    timer = setTimeout((): void => {
      reject(new Error(`Operation timed out after ${timeoutMs}ms`));
    }, timeoutMs);
  });

  try {
    return await Promise.race([promise, timeout]);
  } finally {
    clearTimeout(timer);
  }
}
```

Promise.race повертає результат того Promise, який завершився першим. finally очищає власний таймер після успіху або помилки. При таймауті вихідна операція продовжується, тому окремий тест перевіряє відсутність автоматичного скасування.

Допоміжні функції та інтерфейс

```
export interface UserProfile {
  id: string;
  name: string;
  email: string;
}


export async function delay(ms: number): Promise<void> {
  if (!Number.isFinite(ms) || ms < 0 || ms > 2_147_483_647) {
    throw new RangeError('ms must be between 0 and 2147483647');
  }

  await new Promise<void>((resolve): void => {
    setTimeout(resolve, ms);
  });
}


export async function fetchUserProfiles(
  userIds: string[],
): Promise<UserProfile[]> {
  const requests: Promise<UserProfile>[] = userIds.map(
    async (id: string): Promise<UserProfile> => {
      const delayMs: number = Math.floor(Math.random() * 101) + 50;
      await delay(delayMs);

      return {
        id,
        name: `User ${id}`,
        email: `user${id}@example.com`,
      };
    },
  );

  return await Promise.all(requests);
}
```

5 Перевірка типів і компіляція

```
npm.cmd run typecheck
npm.cmd run build
Get-ChildItem dist -Name
```

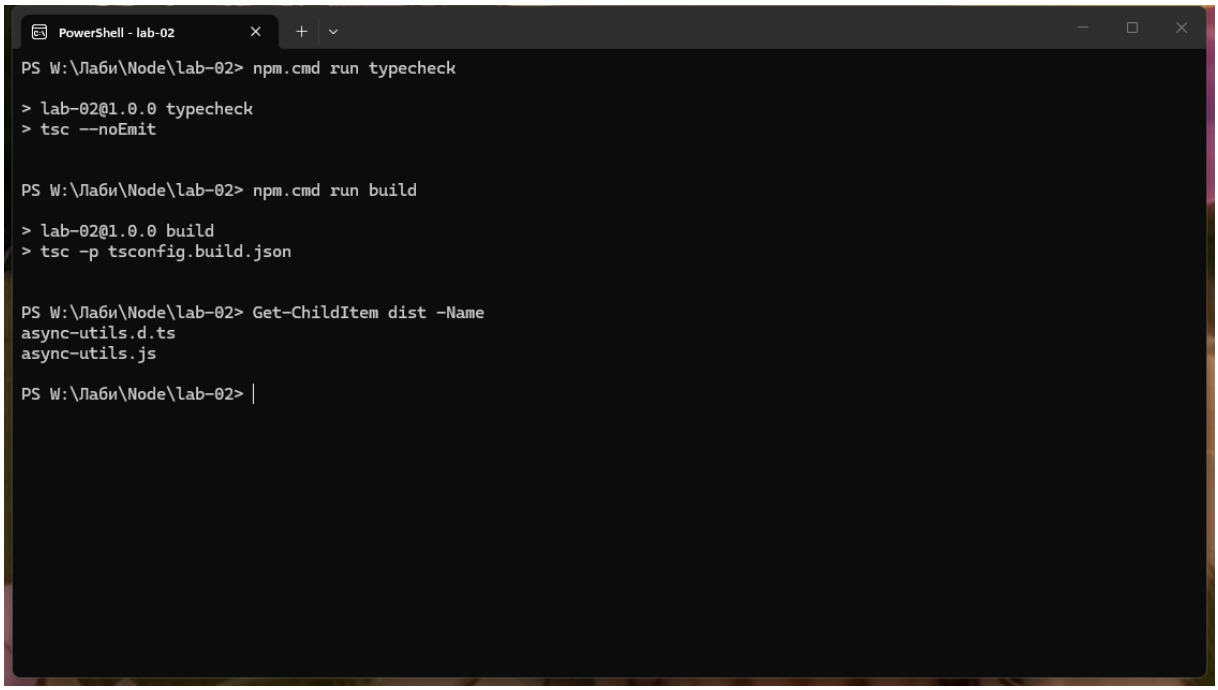


Рис. 4. Успішна перевірка типів і створення JavaScript та декларацій

typecheck перевірив вихідний код і всі тести без помилок. build створив async-utils.js та async-utils.d.ts. Декларація .d.ts містить сигнатури модуля, включно з generics, для використання з іншого TypeScript-коду.

У test/types.ts визначено допоміжний тип Equal. У тестах він перевіряє точний виведений тип результату. Якщо результат стане unknown або іншим типом, присвоєння true такій перевірці не пройде компіляцію.

```
export type Equal<A, B> =
  (<T>() => T extends A ? 1 : 2) extends
  (<T>() => T extends B ? 1 : 2) ? true : false;


const result = await retryOperation(operation, 3);
const inferred: Equal<typeof result, number> = true;
expect(inferred).toBe(true);
```

У виклику retryOperation не зазначено <number>. Тип виводиться з типізованої operation. Аналогічні перевірки в інших наборах підтверджують string[] для перетворення партій та тип об’єкта для raceWithTimeout.

6 Організація асинхронних тестів

Кожен файл містить один describe-блок для відповідної функції. beforeEach вмикає fake timers. afterEach прибирає залишені таймери, повертає реальні таймери та відновлює spyOn-підміни. Завдяки цьому один тест не впливає на наступний.

У тестах retryOperation і processInBatches console.log підмінено через jest.spyOn. Вивід не засмічує результати Jest, а збережені виклики дозволяють перевірити точний текст і порядок повідомлень.

jest.advanceTimersByTimeAsync керовано пересуває час і обробляє пов'язані Promise. Тести перевіряють межі, наприклад 99 мс і ще 1 мс, без очікування справжніх затримок. jest.runAllTimersAsync виконує всю чергу таймерів, коли точна проміжна мить не потрібна [3].

Для Promise, який має відхилитися, перевірка .rejects створюється до пересування часу. Інакше відхилення може виникнути до приєднання обробника й спричинити unhandled rejection.

```
const pending = retryOperation(operation, 2);
const assertion = expect(pending).rejects.toBe(last);
await jest.runAllTimersAsync();
await assertion;
```

Матчер	**Застосування в роботі**
toBe	Точне значення, кількість активних партій або тотожність помилки.
toEqual	Поля профілів, масиви результатів, порядок логування.
toHaveLength	Кількість профілів.
toThrow	Тип або повідомлення відхилення Promise.
toHaveBeenCalledTimes	Кількість спроб та викликів processor.
toHaveBeenLastCalledWith	Аргументи останньої неповної партії.
not.toHaveBeenCalled	Відсутність передчасного виконання або зайвих викликів.

7 Тести функції delay

Перевірено завершення саме після заданої затримки: до 100 мс callback ще не викликано, а після досягнення межі він виконується один раз. Окремі випадки перевіряють нуль та некоректні аргументи. it.each розгортає чотири значення в чотири самостійні тести.

Лістинг delay.test.ts

```
import { delay } from '../src/async-utils';

describe('delay', () => {
  beforeEach(() => {
    jest.useFakeTimers();
  });

  afterEach(() => {
    jest.clearAllTimers();
    jest.useRealTimers();
    jest.restoreAllMocks();
  });


it('завершується після заданої затримки, а не раніше', async () => {
    const complete = jest.fn();
    const pending = delay(100).then(complete);
    expect(pending).toBeInstanceOf(Promise);
    await jest.advanceTimersByTimeAsync(99);
    expect(complete).not.toHaveBeenCalled();
    await jest.advanceTimersByTimeAsync(1);
    await pending;
    expect(complete).toHaveBeenCalledTimes(1);
    expect(complete).toHaveBeenCalledWith(undefined);
  });


  it('підтримує нульову затримку', async () => {
    const pending = delay(0);
    await jest.advanceTimersByTimeAsync(0);
    await expect(pending).resolves.toBeUndefined();
    expect(jest.getTimerCount()).toBe(0);
  });


  it.each([-1, NaN, Infinity, 2_147_483_648])(
    'відхиляє некоректну затримку %s',
    async (ms: number) => {
      await expect(delay(ms)).rejects.toThrow(RangeError);
      expect(jest.getTimerCount()).toBe(0);
    },
  );
});


npm.cmd test -- --runTestsByPath test/delay.test.ts --verbose
```

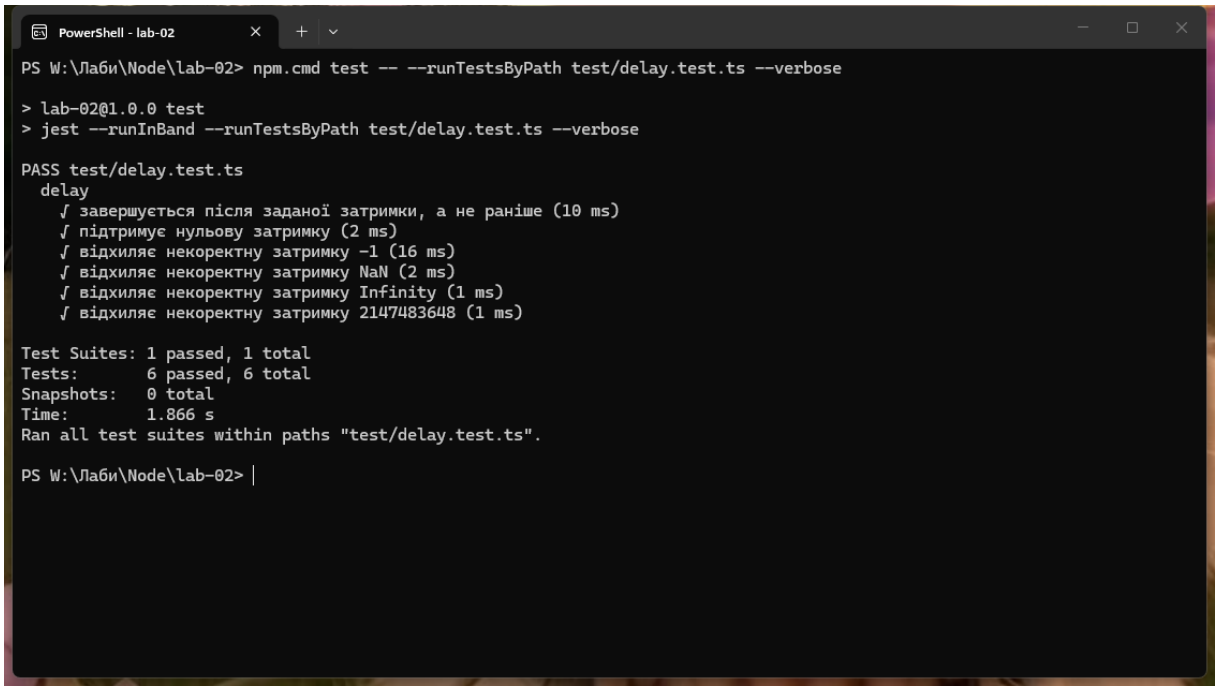


Рис. 5. Результат набору delay у PowerShell

Усі 6 перевірок успішні. Для некоректних значень Promise відхиляється з RangeError ще до створення таймера.

8 Тести функції fetchUserProfiles

Перевірено поля id, name, email; порожній список; паралельний старт; порядок результатів і повторювані id. Для детермінованої перевірки Math.random підмінено так, щоб затримки дорівнювали 150, 50 і 100 мс. Найпершим завершується другий запит, але вихідний порядок має залишитися 9, 2, 4.

Лістинг fetchUserProfiles.test.ts

```
import { fetchUserProfiles, type UserProfile } from '../src/async-utils';

describe('fetchUserProfiles', () => {
  beforeEach(() => {
    jest.useFakeTimers();
  });

  afterEach(() => {
    jest.clearAllTimers();
    jest.useRealTimers();
    jest.restoreAllMocks();
  });


it('повертає профілі з правильними полями', async () => {
    const pending = fetchUserProfiles(['2', '5']);
    await jest.runAllTimersAsync();
    const profiles: UserProfile[] = await pending;
    expect(profiles).toHaveLength(2);
    expect(profiles).toEqual([
      { id: '2', name: 'User 2', email: 'user2@example.com' },
      { id: '5', name: 'User 5', email: 'user5@example.com' },
    ]);
    expect(profiles[0]).toHaveProperty('email', 'user2@example.com');
  });
```

```
  it('повертає порожній масив без таймерів', async () => {
    await expect(fetchUserProfiles([])).resolves.toEqual([]);
    expect(jest.getTimerCount()).toBe(0);
  });


  it('стартує паралельно і зберігає порядок за різних затримок', async () => {
    jest.spyOn(Math, 'random')
      .mockReturnValueOnce(0.999999)
      .mockReturnValueOnce(0)
      .mockReturnValueOnce(0.5);
    const timerSpy = jest.spyOn(globalThis, 'setTimeout');
    const complete = jest.fn();
    const pending = fetchUserProfiles(['9', '2', '4']);
    void pending.then(complete);
    expect(timerSpy).toHaveBeenCalledTimes(3);
    expect(timerSpy.mock.calls.map(call => call[1])).toEqual([150, 50, 100]);
    await jest.advanceTimersByTimeAsync(149);
    expect(complete).not.toHaveBeenCalled();
    await jest.advanceTimersByTimeAsync(1);
    const profiles = await pending;
    expect(profiles.map(profile => profile.id)).toEqual(['9', '2', '4']);
    expect(complete).toHaveBeenCalledTimes(1);
  });


  it('зберігає повторювані ідентифікатори', async () => {
    const pending = fetchUserProfiles(['3', '3']);
    await jest.runAllTimersAsync();
    const profiles = await pending;
    expect(profiles).toHaveLength(2);
    expect(profiles[0]).toEqual(profiles[1]);
  });
});


npm.cmd test -- --runTestsByPath test/fetchUserProfiles.test.ts --verbose
```

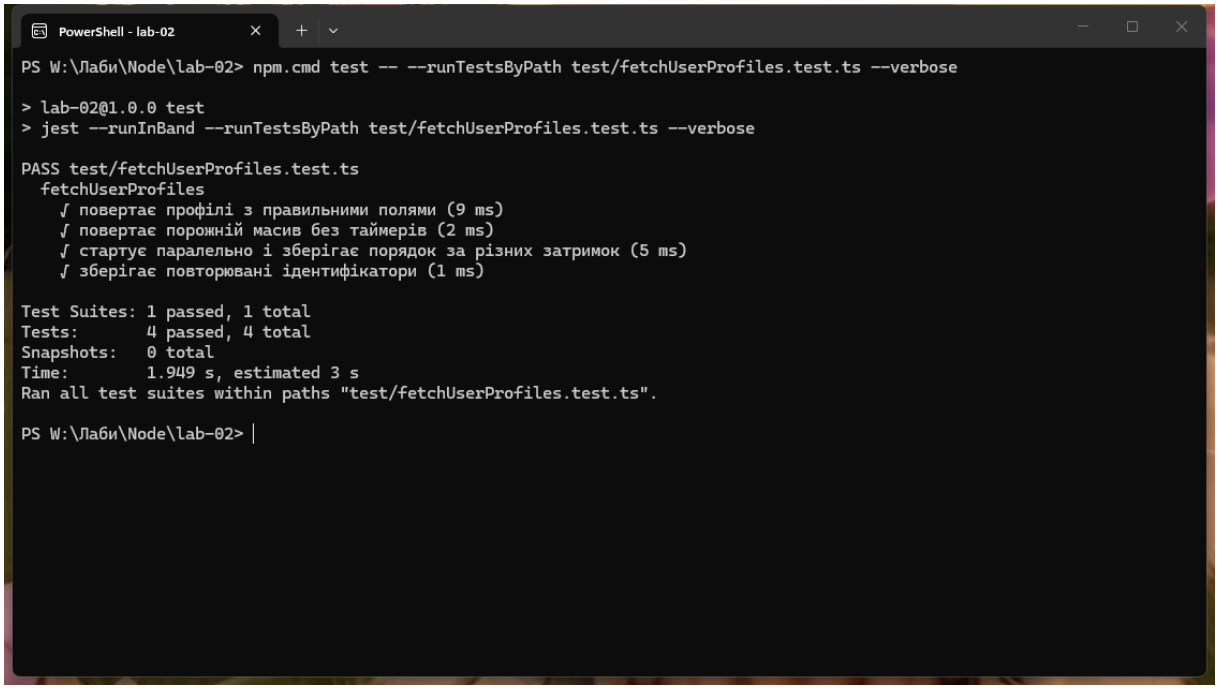


Рис. 6. Результат набору fetchUserProfiles у PowerShell

Усі 4 перевірки успішні. Одразу створюються три таймери, сукупний результат ще не готовий на 149 мс і готовий на 150 мс. Це перевіряє паралельність без залежності від випадкових значень або навантаження комп'ютера.

9 Тести функції retryOperation

Перевірено перший успіх, повтори через 100 мс, припинення після успіху, значення maxRetries за замовчуванням, останню помилку, одну дозволену спробу та п'ять некоректних значень. Типізований mock задає тип повернення Promise, а Equal перевіряє автоматичне виведення number.

Лістинг retryOperation.test.ts

```
import { retryOperation } from '../src/async-utils';
import type { Equal } from './types';

describe('retryOperation', () => {
  let consoleSpy: jest.SpyInstance;

  beforeEach(() => {
    jest.useFakeTimers();
    consoleSpy = jest.spyOn(console, 'log').mockImplementation(() => {});
  });

  afterEach(() => {
    jest.clearAllTimers();
    jest.useRealTimers();
    jest.restoreAllMocks();
  });


it('повертає результат першої спроби з виведеним типом number', async () => {
    const operation = jest.fn<Promise<number>, []>().mockResolvedValue(42);
    const result = await retryOperation(operation, 3);
    const inferred: Equal<typeof result, number> = true;
    expect(inferred).toBe(true);
    expect(result).toBe(42);
    expect(operation).toHaveBeenCalledTimes(1);
    expect(jest.getTimerCount()).toBe(0);
  });


  it('повторює через 100 мс та зупиняється після успіху', async () => {
    const operation = jest.fn<Promise<string>, []>()
      .mockRejectedValueOnce(new Error('first'))
      .mockRejectedValueOnce(new Error('second'))
      .mockResolvedValue('OK');
    const pending = retryOperation(operation, 5);
    const assertion = expect(pending).resolves.toBe('OK');
    await jest.advanceTimersByTimeAsync(99);
    expect(operation).toHaveBeenCalledTimes(1);
    await jest.advanceTimersByTimeAsync(1);
    expect(operation).toHaveBeenCalledTimes(2);
    await jest.advanceTimersByTimeAsync(99);
    expect(operation).toHaveBeenCalledTimes(2);
    await jest.advanceTimersByTimeAsync(1);
    await assertion;
    expect(operation).toHaveBeenCalledTimes(3);
    expect(consoleSpy.mock.calls).toEqual([
      ['Спроба 1...'], ['Спроба 2...'], ['Спроба 3...'],
    ]);
    expect(jest.getTimerCount()).toBe(0);
  });
```

```
  it('без maxRetries виконує рівно три невдалі спроби', async () => {
    const error = new Error('Unavailable');
    const operation = jest.fn<Promise<string>, []>().mockRejectedValue(error);
    const pending = retryOperation(operation);
    const assertion = expect(pending).rejects.toBe(error);
    await jest.runAllTimersAsync();
    await assertion;
    expect(operation).toHaveBeenCalledTimes(3);
    expect(jest.getTimerCount()).toBe(0);
  });


  it('після вичерпання спроб передає саме останню помилку', async () => {
    const last = new Error('Last failure');
    const operation = jest.fn<Promise<number>, []>()
      .mockRejectedValueOnce(new Error('First failure'))
      .mockRejectedValueOnce(last);
    const pending = retryOperation(operation, 2);
    const assertion = expect(pending).rejects.toBe(last);
    await jest.runAllTimersAsync();
    await assertion;
    expect(operation).toHaveBeenCalledTimes(2);
  });


  it('при одній спробі зберігає довільну причину відхилення', async () => {
    const operation = jest.fn<Promise<never>, []>().mockRejectedValue('failed');
    await expect(retryOperation(operation, 1)).rejects.toBe('failed');
    expect(operation).toHaveBeenCalledTimes(1);
    expect(jest.getTimerCount()).toBe(0);
  });


  it.each([0, -1, 1.5, NaN, Infinity])(
    'відхиляє некоректну кількість спроб %s',
    async (maxRetries: number) => {
      const operation = jest.fn<Promise<number>, []>().mockResolvedValue(1);
      await expect(retryOperation(operation, maxRetries)).rejects.toThrow(RangeError);
      expect(operation).not.toHaveBeenCalled();
    },
  );
});


npm.cmd test -- --runTestsByPath test/retryOperation.test.ts --verbose
```

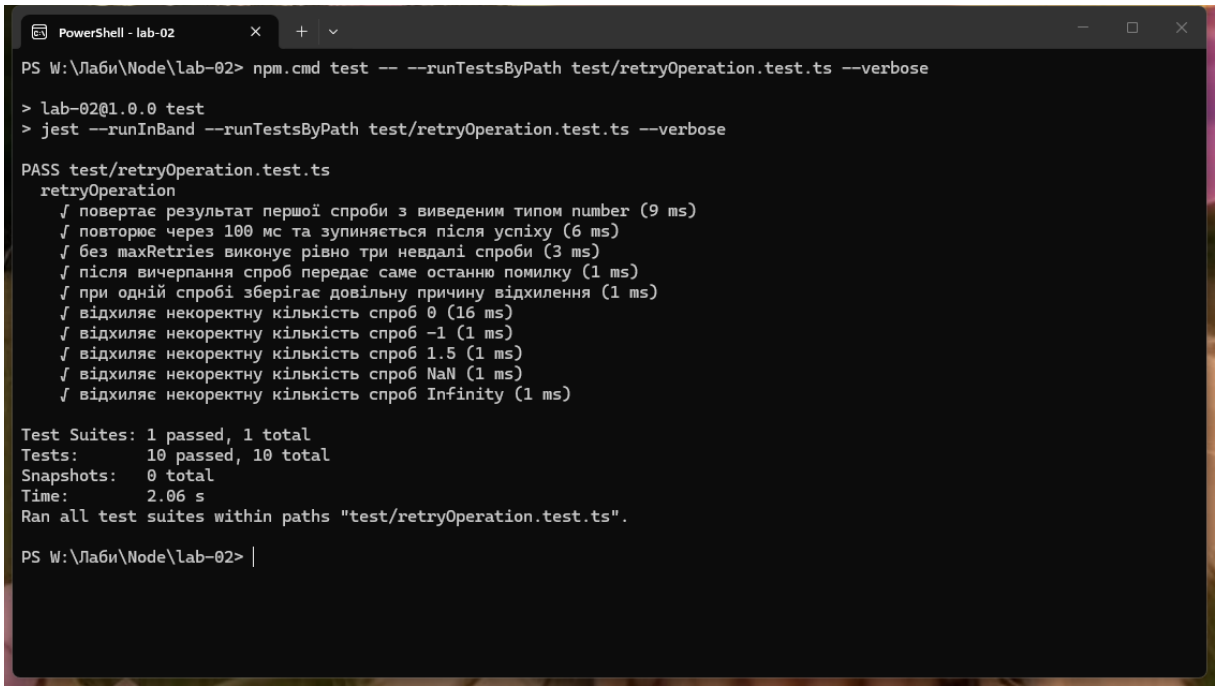


Рис. 7. Результат набору retryOperation у PowerShell

Усі 10 перевірок успішні. Без явного maxRetries невдала operation викликається рівно тричі. Для двох спроб назовні виходить саме останній об’єкт Error. Після завершення повтори та додаткові таймери не залишаються.

10 Тести функції processInBatches

Перевірено перетворення number у string, послідовність, неповну партію, збереження вхідного масиву, порожні дані, великий batchSize, об’єкти на вході, порожній результат processor та зупинку після помилки. П’ять випадків it.each перевіряють валідацію batchSize.

Лістинг processInBatches.test.ts

```
import { delay, processInBatches } from '../src/async-utils';
import type { Equal } from './types';

describe('processInBatches', () => {
  let consoleSpy: jest.SpyInstance;

  beforeEach(() => {
    jest.useFakeTimers();
    consoleSpy = jest.spyOn(console, 'log').mockImplementation(() => {});
  });

  afterEach(() => {
    jest.clearAllTimers();
    jest.useRealTimers();
    jest.restoreAllMocks();
  });
```

```
it('виводить тип string[] при перетворенні number у string', async () => {
    const processor = jest.fn<Promise<string[]>, [number[]]>(
      async batch => batch.map(value => `item-${value}`),
    );
    const result = await processInBatches([1, 2, 3], 2, processor);
    const inferred: Equal<typeof result, string[]> = true;
    expect(inferred).toBe(true);
    expect(result).toEqual(['item-1', 'item-2', 'item-3']);
    expect(processor).toHaveBeenNthCalledWith(1, [1, 2]);
    expect(processor).toHaveBeenLastCalledWith([3]);
  });


  it('обробляє всі партії послідовно та не змінює вхідний масив', async () => {
    const items = [1, 2, 3, 4, 5, 6, 7, 8];
    let active = 0;
    let maxActive = 0;
    const processor = jest.fn<Promise<number[]>, [number[]]>(async batch => {
      maxActive = Math.max(maxActive, ++active);
      await delay(100);
      active--;
      return batch.map(value => value * 2);
    });
    const pending = processInBatches(items, 3, processor);
    expect(processor).toHaveBeenCalledTimes(1);
    await jest.advanceTimersByTimeAsync(99);
    expect(processor).toHaveBeenCalledTimes(1);
    await jest.advanceTimersByTimeAsync(1);
    expect(processor).toHaveBeenCalledTimes(2);
    await jest.runAllTimersAsync();
    await expect(pending).resolves.toEqual([2, 4, 6, 8, 10, 12, 14, 16]);
    expect(processor).toHaveBeenCalledTimes(3);
    expect(processor).toHaveBeenLastCalledWith([7, 8]);
    expect(maxActive).toBe(1);
    expect(items).toEqual([1, 2, 3, 4, 5, 6, 7, 8]);
    expect(consoleSpy.mock.calls).toEqual([
      ['Обробка партії 1/3...'],
      ['Обробка партії 2/3...'],
      ['Обробка партії 3/3...'],
    ]);
  });


  it('не викликає processor для порожнього масиву', async () => {
    const processor = jest.fn<Promise<number[]>, [number[]]>();
    await expect(processInBatches([], 2, processor)).resolves.toEqual([]);
    expect(processor).not.toHaveBeenCalled();
    expect(consoleSpy).not.toHaveBeenCalled();
  });


  it('підтримує партію більшу за масив і об’єкти на вході', async () => {
    const items = [{ name: 'Anna' }, { name: 'Ivan' }];
    const result = await processInBatches(items, 10, async batch =>
      batch.map(item => item.name),
    );
    expect(result).toEqual(['Anna', 'Ivan']);
    expect(result).toContain('Ivan');
    expect(consoleSpy).toHaveBeenCalledTimes(1);
  });


  it('підтримує порожній результат processor', async () => {
    const processor = jest.fn<Promise<string[]>, [number[]]>().mockResolvedValue([]);
    await expect(processInBatches([1, 2], 1, processor)).resolves.toEqual([]);
    expect(processor).toHaveBeenCalledTimes(2);
  });
```

```
  it('після помилки зупиняє наступні партії', async () => {
    const error = new Error('Batch failed');
    const processor = jest.fn<Promise<number[]>, [number[]]>()
      .mockResolvedValueOnce([10])
      .mockRejectedValueOnce(error);
    await expect(processInBatches([1, 2, 3], 1, processor)).rejects.toBe(error);
    expect(processor).toHaveBeenCalledTimes(2);
    expect(processor).not.toHaveBeenCalledWith([3]);
  });


  it.each([0, -1, 1.5, NaN, Infinity])(
    'відхиляє некоректний розмір партії %s',
    async (batchSize: number) => {
      const processor = jest.fn<Promise<number[]>, [number[]]>();
      await expect(processInBatches([1], batchSize, processor)).rejects.toThrow(RangeError);
      expect(processor).not.toHaveBeenCalled();
    },
  );
});
```

```
npm.cmd test -- --runTestsByPath test/processInBatches.test.ts --verbose
```

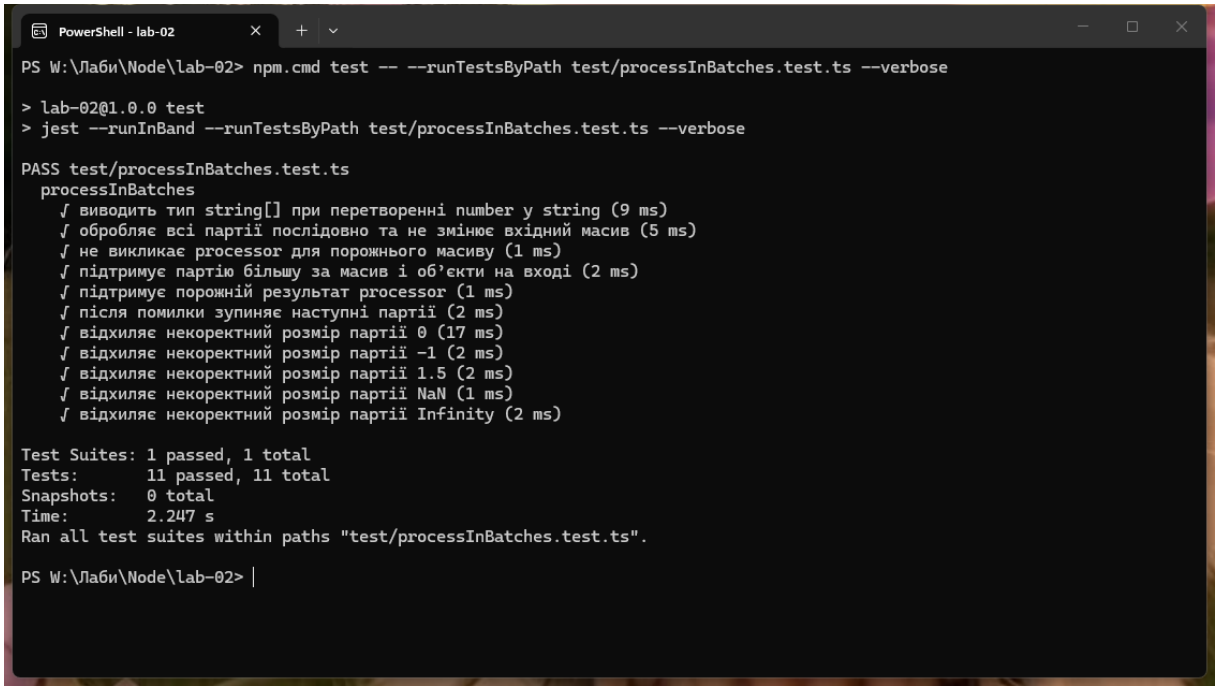


Рис. 8. Результат набору processInBatches у PowerShell

Усі 11 перевірок успішні. Для восьми чисел і batchSize = 3 остання партія містить [7, 8]. Максимальна кількість активних processor дорівнює 1. Якщо друга партія відхиляється, третя не запускається.

11 Тести функції raceWithTimeout

Перевірено швидке повернення об'єкта зі збереженням типу, повідомлення таймауту, початкову помилку операції, очищення таймера, відсутність скасування, нульовий ліміт та чотири некоректні значення. Обробник .rejects приєднується перед просуванням часу.

Лістинг raceWithTimeout.test.ts

```
import { delay, raceWithTimeout } from '../src/async-utils';
import type { Equal } from './types';

describe('raceWithTimeout', () => {
  beforeEach(() => {
    jest.useFakeTimers();
  });

  afterEach(() => {
    jest.clearAllTimers();
    jest.useRealTimers();
    jest.restoreAllMocks();
  });


it('повертає швидкий результат з автоматично виведеним типом', async () => {
    const value = { id: 1, name: 'Anna' };
    const pending = raceWithTimeout(delay(50).then(() => value), 100);
    await jest.advanceTimersByTimeAsync(50);
    const result = await pending;
    const inferred: Equal<typeof result, typeof value> = true;
    expect(inferred).toBe(true);
    expect(result).toBe(value);
    expect(result).toMatchObject({ id: 1 });
    expect(jest.getTimerCount()).toBe(0);
  });


  it('відхиляється з точним повідомленням після перевищення ліміту', async () => {
    const pending = raceWithTimeout(delay(200), 100);
    const assertion = expect(pending).rejects.toThrow(/^Operation timed out after 100ms$/);
    await jest.advanceTimersByTimeAsync(100);
    await assertion;
    expect(jest.getTimerCount()).toBe(1);
    await jest.runAllTimersAsync();
  });


  it('передає початкову помилку й очищає таймер', async () => {
    const error = new Error('Network unavailable');
    await expect(raceWithTimeout(Promise.reject(error), 100)).rejects.toBe(error);
    expect(jest.getTimerCount()).toBe(0);
  });


  it('таймаут не скасовує вже запущену операцію', async () => {
    const complete = jest.fn();
    const operation = delay(200).then(complete);
    const assertion = expect(raceWithTimeout(operation, 100)).rejects.toThrow(Error);
    await jest.advanceTimersByTimeAsync(100);
    await assertion;
    expect(complete).not.toHaveBeenCalled();
    await jest.advanceTimersByTimeAsync(100);
    await operation;
    expect(complete).toHaveBeenCalledTimes(1);
  });


  it('приймає нульовий ліміт для вже виконаного Promise', async () => {
    await expect(raceWithTimeout(Promise.resolve('ready'), 0)).resolves.toBe('ready');
    expect(jest.getTimerCount()).toBe(0);
  });
```

```
  it.each([-1, NaN, Infinity, 2_147_483_648])(
    'відхиляє некоректний ліміт %s',
    async (timeoutMs: number) => {
      await expect(raceWithTimeout(Promise.resolve(1),
timeoutMs)).rejects.toThrow(RangeError);
      expect(jest.getTimerCount()).toBe(0);
    },
  );
});
```

```
npm.cmd test -- --runTestsByPath test/raceWithTimeout.test.ts --verbose
```

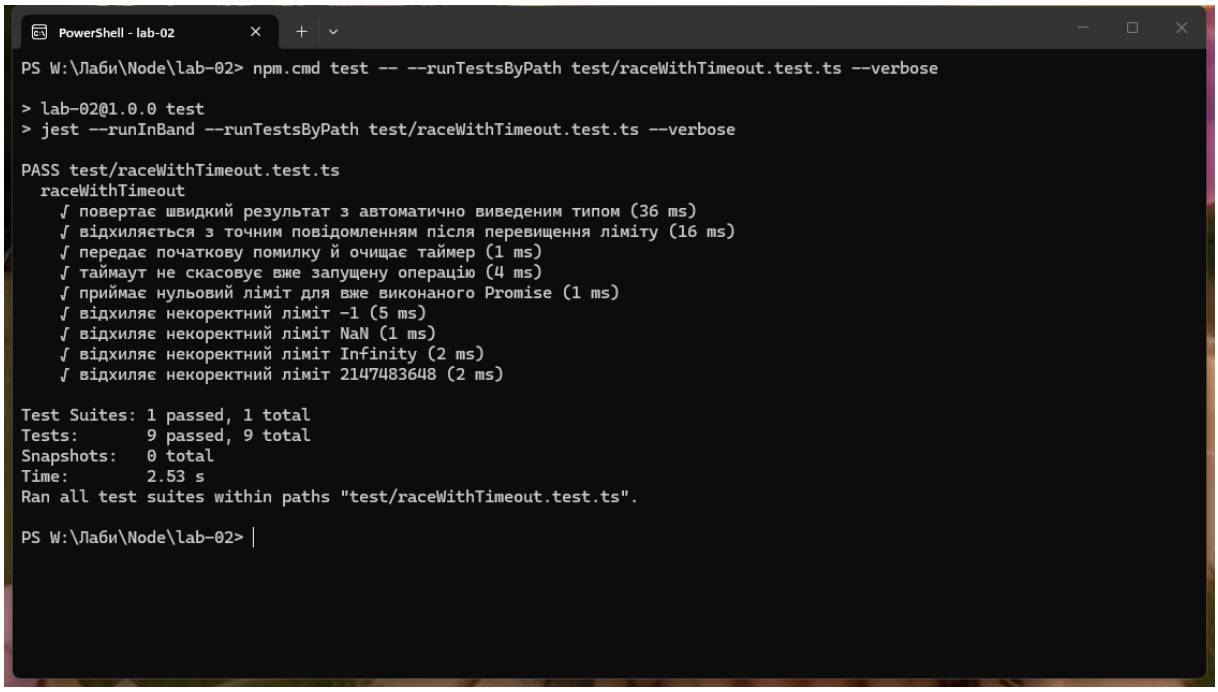


Рис. 9. Результат набору raceWithTimeout у PowerShell

Усі 9 перевірок успішні. Швидка операція повертає початковий об’єкт; при перевищенні 100 мс виникає повідомлення Operation timed out after 100ms. Після таймауту залишається лише таймер повільної операції, яка згодом завершується.

12 Загальний запуск тестів

```
npm.cmd test
```

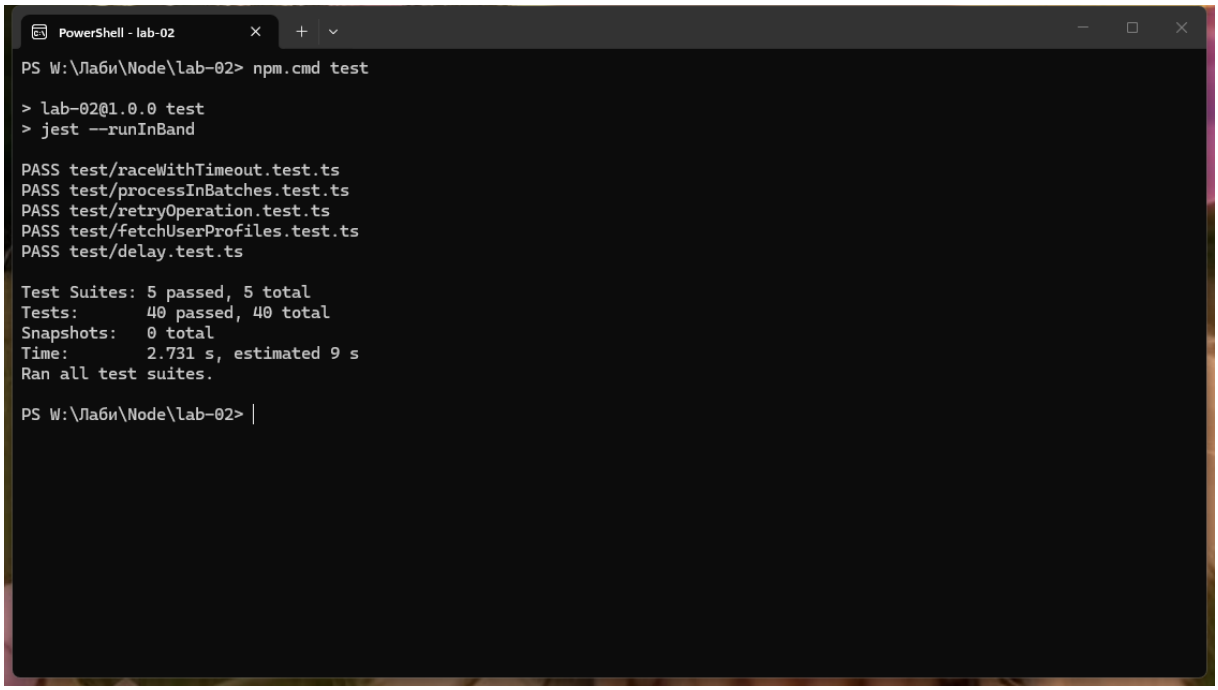


Рис. 10. Загальний запуск п’яти наборів Jest

Набір	**Мінімум за завданням**	**Виконано**
delay	Додаткова перевірка	6
fetchUserProfiles	3	4
retryOperation	3	10
processInBatches	3	11
raceWithTimeout	2	9
Разом	11 для чотирьох основних функцій	40

Загальний запуск завершився без помилок: Test Suites 5 passed, Tests 40 passed. Snapshot-тести не використовуються; значення, помилки, порядок викликів і часові межі перевіряються явними твердженнями expect.

13 Вимірювання покриття коду

```
npm.cmd run test:coverage
```

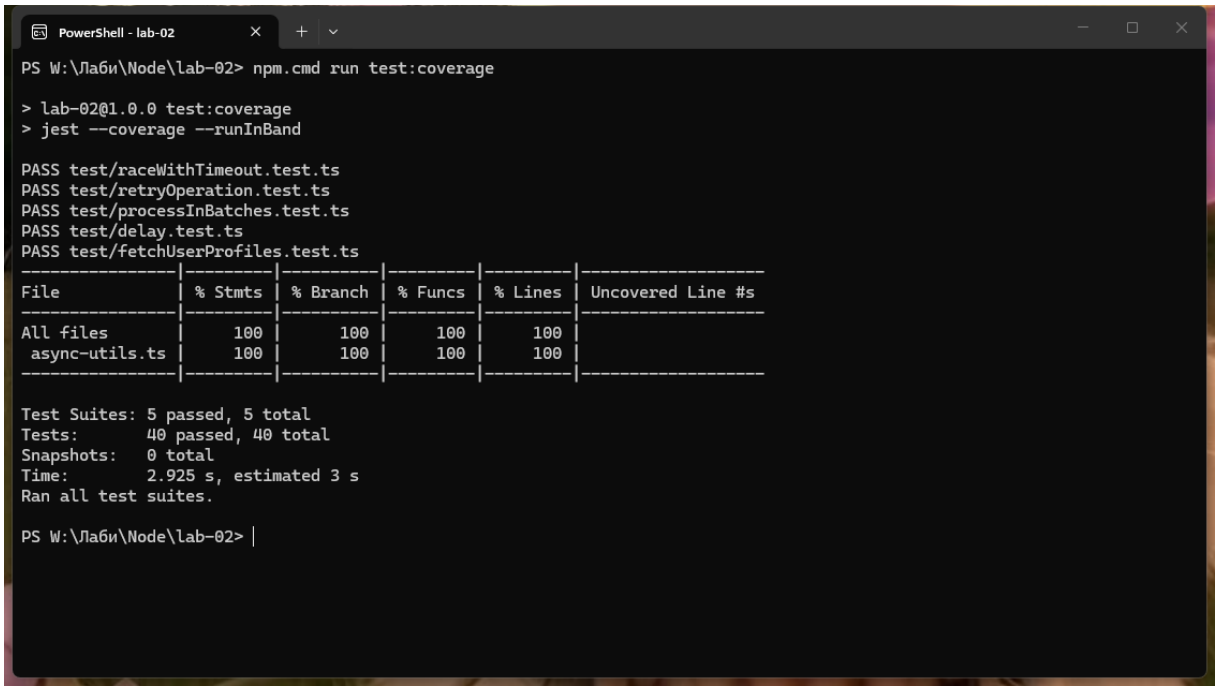


Рис. 11. Тести та таблиця стовідсоткового покриття

Метрика	**Покрито**	**Усього**	**Відсоток**
Інструкції	45	45	100%
Гілки	21	21	100%
Функції	9	9	100%
Рядки	43	43	100%

Усі чотири метрики становлять 100%. Область покриття включає весь вихідний модуль src/async-utils.ts; виключень для окремих функцій чи рядків немає. Для покриття гілок перевірено стандартні параметри, валідацію, обидва завершення retry, порожній цикл партій та помилки асинхронних операцій.

Показник functions враховує також внутрішні callback-функції, тому його кількість більша за п’ять експортованих функцій. Інтерфейси, generics та інші суто типові конструкції не виконуються у JavaScript; їхню правильність перевіряє TypeScript.

Після запуску у coverage/index.html доступний HTML-звіт, у coverage/lcov.info — дані LCOV, а coverage-summary.json містить числовий підсумок. Ці файли можна згенерувати повторно.

100% покриття підтверджує виконання всіх врахованих ділянок коду, але саме по собі не доводить відсутності будь-яких помилок. Тому перевірки також порівнюють конкретні результати, порядок, число викликів та межі часу. Fake timers перевіряють планування, а не реальну швидкодію програми.

Висновок:

Під час лабораторної роботи налаштовано Jest із ts-jest для тестування асинхронного TypeScript-коду. Функції retryOperation, processInBatches та raceWithTimeout узагальнено через generics без зміни виконуваної логіки. Автоматичне виведення типів результатів перевірено компілятором.

Створено 40 тестів у п’яти describe-блоках. Використано async/await, .resolves, .rejects, типізовані jest.fn, jest.spyOn, beforeEach, afterEach та керовані таймери. Перевірено правильні результати, помилки, граничні аргументи, послідовність виконання та очищення ресурсів.

Усі тести пройшли, а покриття інструкцій, гілок, функцій та рядків вихідного модуля становить 100%. Команди перевірки типів і компіляції також завершилися без помилок.